

Hoja 1: Introducción a los Interfaces del Sistema

Objetivos

Estudiar los principales interfaces para interactuar con el sistema operativo:

- **Interfaz de línea de comandos**, se revisan los conceptos básicos del manejo de un sistema operativo UNIX. El objetivo principal será el manejo de la *shell* y sus características. Además, se verán algunas de las herramientas básicas y una breve introducción a la programación en *shell script*.
- **API de un sistema Unix** y su entorno de desarrollo. En particular, se estudiarán las convenciones de llamadas a la API del sistema y funciones específicas para la gestión de errores y la obtención de información de usuarios.

Ejercicios Prácticos

Todos los ejercicios de esta hoja son prácticos y requieren el entorno de usuario básico (shell) y de desarrollo (compilador, editores y depurador). Este entorno está disponible en las máquinas virtuales de la asignatura, el entorno docker, y en la máquina física del laboratorio.

Interfaz de línea de comandos

La *shell* bash

Páginas de manual

Ejercicio 1. Consultar la página de manual de `man` (`man man`). Especialmente las secciones de una página de manual (NAME, SYNOPSIS, DESCRIPTION...).

Ejercicio 2. En documentos técnicos relacionados con linux, es común utilizar la sintaxis `comando(n)` para indicar que nos estamos refiriendo al comando que se encuentra descrito en la sección `n` del manual. Por ejemplo, `uname(1)` se refiere al comando `uname` de la shell, mientras que `uname(2)` se refiere a la función `int uname(struct utsname*)` de la librería estándar de C.

Para buscar en una sección determinada, simplemente se puede especificar como `man sección nombre`. Por ejemplo, revisa la diferencia entre `man 1 time` y `man 2 time`, o entre `man 1 uname` y `man 2 uname`. Como recordatorio, las secciones del manual son:

- **Sección 1:** Comandos de usuario (ejecutables o *scripts* de shell)
- **Sección 2:** Llamadas al sistema (funciones provistas por el kernel)
- **Sección 3:** Funciones de librería (por ejemplo, de la librería C estándar)
- **Sección 4:** Ficheros especiales (normalmente en `/dev`)
- **Sección 5:** Formatos de fichero y convenciones
- **Sección 6:** Juegos
- **Sección 7:** *Overviews*, convenciones y misceláneo
- **Sección 8:** Comandos de administración del sistema (normalmente solo root)
- **Sección 9:** Rutinas del kernel (poco frecuente, específico de cada sistema)

Ejercicio 3. Para buscar páginas de manual sobre un tema en particular se puede usar la opción `-k`, o las órdenes `whatis` o `apropos`. Buscar las órdenes relacionadas con la "hora" (*time*). **Nota:** Este comando accede a una base de datos sencilla para cada página de manual. La base de datos se

gestiona con el comando `mandb`.

La línea de comandos

Ejercicio 1. El *prompt* de la shell es el texto que aparece cada vez que la shell se encuentra a la espera de que el usuario escriba un comando a ejecutar. Arrancar un terminal y comprobar el *prompt* de la shell. Cada usuario tiene un *prompt* distinto configurable.

Para cambiar de usuario podemos usar el programa `sudo(8)`. Ejecutar `sudo -i` y comprobar si cambia el *prompt*, regresar al usuario inicial con la orden `exit`. El comando `exit` termina la ejecución de la shell y devuelve opcionalmente un código de retorno. Comprobar también qué información dan los comandos `id(1)` (identidad del usuario y sus grupos) y `whoami(1)` (nombre del usuario actual), y en qué se diferencian.

Comandos y secuencias de comandos

Ejercicio 1. Estudiar el funcionamiento del comando `echo(1)`. ¿Qué hacen las opciones `-n` y `-e`? Imprimir la frase “Curso de Sistemas Operativos” con un tabulador al principio.

Ejercicio 2. Los comandos se pueden partir en varias líneas terminando cada una con `\`. Imprimir la frase del ejemplo anterior, escribiendo cada palabra en una línea. Comprobar que el *prompt* de la shell cambia en el modo multilinea.

Ejercicio 3. La shell dispone de una serie de *meta-caracteres* que permiten controlar su comportamiento. En especial: `|`, `&&`, `;`, `( )`, `|`, `&`, sirven para generar secuencias o listas de comandos. Estudiar el funcionamiento de las siguientes secuencias:

```
echo línea uno; echo línea dos; echo línea tres
```

```
echo línea uno && echo línea dos && echo línea tres
```

```
echo línea uno | echo línea dos; echo línea tres
```

```
(echo En una sub-shell; exit 0) && echo acabó bien | echo acabó mal
```

```
(echo En una sub-shell; exit 1) && echo acabó bien | echo acabó mal
```

Como recordatorio:

- `;`: ejecuta el siguiente comando pase lo que pase (no depende del resultado del anterior)
- `&&`: ejecuta el siguiente comando solo si el anterior terminó con éxito (código 0)
- `|`: ejecuta el siguiente comando solo si el anterior falló (código distinto de 0)

`exit 0` indica que el programa/script terminó correctamente; cualquier otro código (por ejemplo `exit 1`) indica que terminó con error. Ese código es el que consultan `&&` y `|` para decidir si continuar.

Control de trabajos

Ejercicio 1. Los comandos se pueden lanzar en segundo plano añadiendo `&` al final. Lanzar `sleep 100 &` y comprobar con `jobs` que aparece en ejecución. Traer el trabajo a primer plano con `fg` y pararlo con `Ctrl+Z`. Comprobar de nuevo con `jobs` que el proceso queda en estado “detenido” (*Stopped*). Reanudar el proceso en segundo plano con el comando `bg`. Finalmente terminarlo con `kill %1`.

Variables de entorno

Ejercicio 1. Consultar el entorno de la shell mediante `env`.

Ejercicio 2. El valor de las variables se puede acceder con `echo` seguido del prefijo `$` y el nombre de la variable. Consultar el valor y determinar el significado de: `USER`, `UID`, `HOME`, `PWD`, `SHELL`, `$`,

PPID, `?`, `PATH`. Por ejemplo:

```
(exit 0); echo $?; (exit 4); echo $?  
echo $$ $PPID
```

Ejercicio 3. Fijar el valor de las variables `VARIABLE1`, `VARIABLE2` y `VARIABLE3` a “Curso”, “de” y “Sistemas Operativos”, respectivamente. Imprimir la frase “Curso de Sistemas Operativos” accediendo a las variables.

Entrar en otra shell, ejecutando `bash`, repetir la orden que imprime la frase. ¿Cuál es el resultado? Hacer que la frase se imprima correctamente en la nueva shell definiendo las variables con el comando `export`.

Ejercicio 4. Las colisiones entre variables se pueden evitar poniendo el nombre de la variable entre llaves `{}`. Fija la variable `VAR1` a “1+1=” y `VAR12` a “error”. ¿Qué imprime `echo $VAR12`? ¿Cómo se imprimiría “1+1=2” (sin espacios) usando el valor de `VAR1`?

Ejercicio 5. La variable `PS1` guarda la estructura del *prompt*. Consultar la sección `PROMPTING` en la página de manual de `bash` y cambiar el *prompt* a “12:39|~%” (hora | directorio %). Para localizar la sección `PROMPTING` dentro de `man bash` (que es una página muy larga), no hace falta recorrerla entera: una vez abierta, pulsar `/` y escribir `PROMPTING`, luego `Enter`. Con `n` se salta a la siguiente coincidencia, y con `N` a la coincidencia anterior.

Ejercicio 6. `PATH` es una de las variables más importantes del entorno de la *shell*. Consultar su valor y añadir el directorio actual al `PATH`. ¿Qué diferencia hay entre añadir `./` y `$PWD`? Recuerda que para concatenar (añadir algo nuevo al `PATH` sin perder lo que ya había) hay que anteponer el valor actual seguido de `:`, por ejemplo: `export PATH=$PATH:.`

Ejercicio 7. La orden `which(1)` determina qué comando se ejecutará cuando se usa en la línea de comandos. Determinar qué fichero se usa para ejecutar `bash`, `cd`, `echo`, y `test`. De forma similar, `type` sirve para identificar cómo la shell interpreta el comando ¿cuáles de los anteriores comandos son *built-in*?

Ejercicio 8. Los *alias* permiten crear atajos para comandos frecuentes. En una terminal interactiva, ejecutar `alias ll="ls -l"` y comprobar que `ll` funciona como `ls -l`. Eliminarlo con `unalias ll`.

Aritmética en la shell

Ejercicio 1. La shell permite hacer operaciones aritméticas con la construcción `$(( ))`. Fijar `a=5` y calcular con esta construcción: `a+3`, `a<<1` (desplazamiento de bits a la izquierda), `a & 3` (AND a nivel de bit) y `a | 8` (OR a nivel de bit). Comprobar cada resultado con `echo(1)`.

Historial de órdenes

Ejercicio 1. `Bash` guarda el historial de órdenes ejecutadas en el fichero indicado por la variable `HISTFILE` (normalmente `~/.bash_history`). En una terminal interactiva, ejecutar varias órdenes y luego pulsar `Ctrl+R` para buscar en el historial escribiendo parte de una de ellas; comprobar que `Bash` la localiza y que se puede volver a ejecutar con `Enter`. Consultar también el comando `history(3)` para listar las últimas órdenes.

Manejo de cadenas y flujos de caracteres

Ejercicio 1. Estudiar el comando `sort(1)` (`man sort`). Ordenar las palabras: zorro, pájaro, vaca, caballo y abeja, imprimiendo cada una en una línea (`\n`) con el comando `echo` y encadenando su salida (tubería `|`) con el comando `sort`.

Ejercicio 2. Escribir las palabras anteriores en un fichero (`texto1`) usando el comando `echo` y la redirección (`>`). Repetir el ejercicio anterior usando en este caso el fichero `texto1` y no la entrada estándar del comando `sort`.

Ejercicio 3. El comando `cat` muestra el contenido de un fichero, probar con `texto1`. Además, si no se especifica un fichero (o se usa `-`, ver `man cat`), recoge la entrada estándar que puede redirigirse a un fichero. Escribir las palabras: pera, manzana, plátano y ciruela, cada una en una línea, en el fichero `texto2`. **Nota:** el flujo de entrada estándar se cierra con `Ctrl+d` (envía el caracter especial EOF). Comprobar el contenido de `texto2` con `cat(1)`.

Ejercicio 4. `cat(1)` se puede usar para concatenar ficheros si se especifican como argumentos. Unir los ficheros `texto1` y `texto2` en `texto3` usando la redirección (`>`).

Ejercicio 5. `wc(1)` sirve para contar palabras, líneas de un fichero de texto y bytes. Comprobar el tamaño de los ficheros `texto1`, `texto2` y `texto3` con `wc`, y compararlo con la salida de `ls -l *`. Consultar la página de manual para ver cómo controlar la salida del comando.

Ejercicio 6. Los comandos `head(1)` y `tail(1)` permiten ver el principio y el final de un fichero. Vamos a usar `dmesg(1)` (mensajes del sistema) para probar el funcionamiento de estos comandos:

- En primer lugar determinar el número de líneas que produce el comando.
- Usar `tail` para quedarse con la última parte (ej. últimas 15 líneas)
- Usar `head` para quedarse con las primeras líneas (ej. primeras 3 líneas)

Nota: `dmesg` requiere permisos para leer el buffer del kernel; si da error de permiso, ejecutar antes `sudo -i` para pasar a root.

Ejercicio 7. El comando `tr` sirve para cambiar caracteres (*translate*) o eliminarlos. Poner todas las palabras del fichero `texto1` en una línea sustituyendo el carácter fin de línea por un tabulador.

El sistema de ficheros

Ejercicio 1. Para cambiar de directorio se usa la orden `cd`. Si se indica un directorio, el comando `cd` cambiará el directorio de trabajo al indicado. Sin embargo, si se ejecuta el comando sin argumentos, se cambiará al directorio del usuario (directorio HOME).

Para averiguar la ruta (*path*) del directorio actual, se puede utilizar el comando `pwd`, o consultar la variable de entorno `PWD`.

- Probar a cambiar al directorio `/usr/bin`, y comprobarlo con `pwd`.
- Volver al directorio anterior (`cd -`).
- Cambiar ahora de nuevo al directorio `/usr/bin` pero de forma relativa desde el directorio de usuario.
- Volver finalmente al directorio de usuario.

Nota: El `.` hace referencia al directorio actual y `..` hace referencia al directorio directamente superior.

Nota 2: Recuerda que bash sustituye el caracter `~` por el directorio del usuario. Por lo tanto, las cuatro siguientes órdenes son similares:

```
cd
cd /home/miUsuario/
```

```
cd %HOME
cd ~
```

Ejercicio 2. La orden `mkdir(1)` sirve para crear directorios. Hacer el directorio 'mis_archivos' en el directorio de trabajo. Comprobar su creación con el comando `ls(1)` (*list*).

Ejercicio 3. La opción `-p` de `mkdir` sirve para crear un directorio y todos aquellos que sean necesarios en la ruta. Crear el directorio `mis_archivos/prueba/texto/tmp`, probar con la opción `-p` y sin ella (`man mkdir`). Para ver de un vistazo todos los ficheros y subdirectorios creados dentro de `mis_archivos`, se puede usar el comando `tree mis_archivos` (si no está instalado, se puede instalar mediante la orden `sudo apt install tree`, o equivalente según la distribución usada).

Ejercicio 4. Un directorio se puede borrar con `rmdir`, pero debe estar vacío. Eliminar el directorio `prueba` (y todos sus contenidos con `rm -r`).

Permisos de ficheros

Ejercicio 1. Ejecutar `ls -l` sobre cualquier fichero y examinar la cadena de permisos (por ejemplo `-rw-r--r--`). Identificar los tres bloques (propietario, grupo, otros) y los tres tipos de permiso en cada uno (lectura `r`, escritura `w`, ejecución `x`).

Ejercicio 2. Crear un fichero vacío con `touch prueba.sh`. Comprobar sus permisos por defecto con `ls -l`. Darle permiso de ejecución con `chmod +x prueba.sh` y comprobar el cambio. Quitar el permiso de escritura al grupo con `chmod g-w prueba.sh`. Finalmente, fijar los permisos exactos a `-rw-r--r--` usando la notación octal (`chmod 644 prueba.sh`).

Ejercicio 3. Los ficheros y directorios se pueden renombrar y también copiar. Copiar el fichero `texto1` a `copia1` con la orden `cp(1)m`. Los ficheros y directorios se pueden renombrar con el comando `mv(1)`. Renombrar `copia1` a `fichero1`.

Ejercicio 4. Crear un directorio llamado 'copia' y copiar en él los archivos `texto1`, `texto2` y `fichero1` con la orden `cp` en una única línea.

Ejercicio 5. Los directorios se pueden copiar usando la opción `-r`. Copiar el directorio `copia` al directorio `otra_copia`. Consultar el propósito de las opciones `-f` y `-i` en la página de manual de `cp`.

Ejercicio 6. Para borrar usar el comando `rm(1)`. Borrar los ficheros del directorio `copia` con `rm`, y finalmente el propio directorio `copia` con `rmdir`. Los comandos `cp` y `rm` aceptan opciones recursivas (`-r`) para copiar y borrar directorios respectivamente. Borrar el directorio `otra_copia` con `rm -r`.

Comodines (globbing)

Ejercicio 1. Además de buscar con `find`, la propia shell puede expandir patrones directamente sobre los nombres de fichero, antes incluso de ejecutar el comando. Sobre un directorio con varios ficheros de prueba, comparar el resultado de: `*` (cualquier secuencia de caracteres), `?` (exactamente un carácter) y `[ab]` (un carácter dentro del conjunto indicado). Por ejemplo, crear los ficheros `a1.txt`, `a2.txt`, `b1.txt` y `b2.log`, y comparar la salida de los comandos `ls *.txt`, `ls ?1*` y `ls [ab]1.txt`.

Búsqueda de ficheros

Ejercicio 1. La forma básica de uso de `find` es `find <ruta_de_búsqueda> -name <patrón_nombre>`. Por ejemplo, buscar todos los ficheros que empiezan por `texto` en `/home`. Recuerda que el patrón, al ser una cadena con posibles comodines, debe ir entre comillas (por ejemplo, `-name "texto*"`) para que lo interprete `find` y no la propia shell.

Ejercicio 2. Se puede además buscar por tipo usando la opción `type`. Consultar la página de manual de `find` para buscar todos los directorios en `$HOME`.

Ejercicio 3. La opción `size` permite buscar por tamaño con los modificadores `c`, `b`, `k`, `M` y `G`. Si se antepone `+` al tamaño se seleccionan ficheros mayores, con `-` se seleccionan ficheros más pequeños. Buscar ficheros mayores de 10M en el sistema.

Ejercicio 4. `find` se combina a menudo con `xargs` para aplicar un comando a cada resultado. Buscar todos los ficheros `texto*` del directorio actual y, en un único paso (usando una tubería `find ... | xargs ...`), contar cuántas líneas tiene cada uno con `wc -l`.

Redirecciones, Tuberías y Expresiones regulares

Ejercicio 1. Ejecutar en el directorio `$HOME` el comando `ls -l text* nada* > salida`. ¿Qué sucede?, ¿cuáles son los contenidos del fichero `salida`? Con el comando anterior redirigir la salida estándar a `salida.out` y la salida estándar de error a `salida.error`.

Ejercicio 2. El operador de redirección (`>`) ‘trunca’ el contenido del fichero. Se puede añadir al contenido ya existente mediante (`>>`). Repetir el comando anterior pero sin borrar el contenido de los ficheros.

Ejercicio 3. Se pueden *duplicar* los descriptores con `>&` y `>>&`, en la forma: `comando > salida 2>&1`. Redirigir la salida estándar y de error del comando `ls` anterior al fichero `salida.txt`. Comprobar que el comportamiento es distinto a `comando 2>&1 > salida` (las duplicaciones se hacen secuencialmente).

Ejercicio 4. Muchas veces no resulta interesante la salida del comando, sólo su resultado (`$?`); esto se consigue redirigiendo las salidas a `/dev/null`. Probar el funcionamiento con las órdenes anteriores.

Ejercicio 5. También es posible redirigir la entrada estándar. Comparar la ejecución de las secuencias:

- `cat texto1 | sort`
- `sort < texto1`

Ejercicio 6. Usando el comando `grep` y una expresión regular adecuada, buscar en el fichero `texto1` las siguientes palabras:

```
$ cat texto1
zorro
pájaro
vaca
caballo
abeja
```

- las palabras con ‘ja’
- las palabras que terminan en ‘ja’
- las palabras que tiene dos aes con una letra en medio (aba,aca,aaa,ala,...)

- el patrón `alo ó allo` (la `l` se repite una o dos veces)

Programación en lenguaje shell

Importante: Para ejecutar un script debe tener permisos de ejecución, que se pueden otorgar con el comando `chmod +x <ruta_del_script>` (ver la sección “Permisos de ficheros” más arriba si no lo tienes claro).

Ejercicio 1. Argumentos de un programa. Escribir un script que muestre el nombre del fichero del script (`$0`), el primer (`$1`) y segundo (`$2`) argumento, cada uno en una línea. Además, la variable `$#` contiene el número de argumentos y `$@` sirve para referirse a todos los argumentos.

Ejercicio 1b. El comando `shift`. Modificar el script del ejercicio anterior para que, después de mostrar `$1` y `$2`, ejecute `shift` y vuelva a mostrar `$1`, `$2` y `$#`. ¿Qué ha pasado con los argumentos?

Ejercicio 2. Sentencias Condicionales. Escribir un programa que acepte exactamente un argumento. Si no es así, debe terminar con código 1 y mostrar el mensaje correspondiente. El argumento se interpretará como una ruta. Si la ruta es un fichero regular (ver `man bash`, sección `CONDITIONAL EXPRESSIONS`), el script mostrará el número de líneas que tiene.

Nota: La ejecución de un programa se puede asignar a una variable que guardará la salida estándar. Además, el comando `cut` puede ser útil para separar una cadena y seleccionar un campo. Cuidado: los corchetes de la sintaxis condicional (`[ ... ]`) llevan espacios obligatorios alrededor tanto de la expresión como de cada corchete (`[ -f "$1" ]`, no `[ -f "$1" ]`); si se omiten, la shell no reconoce `[` como el comando que es y el script falla.

```
$ ./ejercicio2.sh /etc/passwd
El fichero /etc/passwd tiene 22 líneas
$ ./ejercicio2.sh /etc/pas
El fichero /etc/pas no existe o no es regular
$ ./ejercicio2.sh
Uso ejercicio2.sh <ruta>
```

Ejercicio 2b. Más condiciones. El ejercicio anterior sólo comprueba si la ruta es un fichero regular (`-f`). Prueba en la terminal (no hace falta un script) el resultado de estas otras condiciones sobre distintos ficheros y directorios de tu sistema: `-d` (es un directorio), `-e` (existe), `-r/-w/-x` (permisos de lectura/escritura/ejecución) y `-s` (no está vacío). Prueba también condiciones de cadenas: `[ "$a" = "$b" ]`, `[ "$a" != "$b" ]`, `[ -z "$a" ]` (cadena vacía) y `[ -n "$a" ]` (cadena no vacía); y condiciones aritméticas: `-eq`, `-ne`, `-gt`, `-ge`, `-lt`, `-le` y la negación `!`. Por ejemplo: `n1=5; n2=10; [ $n1 -lt $n2 ] && echo "n1 es menor"`.

Ejercicio 3. Bucles. Escribir un *script shell* que itere por cada fichero de un directorio dado (argumento del script) y si es un fichero regular, muestre en una línea el nombre y el número de palabras.

Ejercicio 3b. Bucle `while`. Escribir un script que reciba un número como argumento y haga una cuenta atrás hasta 0 usando `while` (condición `[ $n -gt 0 ]`) y la resta aritmética `n=$(( n-1 ))` en cada iteración.

Ejercicio 4. Funciones. Para escribir una función en `bash` se usa la siguiente sintaxis. Probar la ejecución del siguiente script. ¿Qué valor se almacena en la variable `A`?

```
#!/bin/bash

function hola(){
    echo "Hola $1!"
}

hola mundo
A=$(hola mundo)
echo "el valor de A es ${A}"
```

Proyecto: Agenda en Shell script.

Escribir un programa que sirva de agenda. El programa almacenará los datos de la agenda en una base de datos en texto plano, cada línea del fichero será un registro con el formato:

nombre:teléfono:dirección de mail

La agenda dará las opciones de *listar*, para mostrar todos los registros formateados; *añadir* un registro, *borrar* un registro identificando por el campo nombre; *buscar* un registro por un patrón aplicado a cualquier campo y *salir* del programa. El siguiente ejemplo muestra una ejecución:

```
$ ./agenda.sh
1)listar
2)buscar
3)borrar
4)añadir
5)salir

#? 4
Nombre:juan
Teléfono:2345
Mail:juan@ucm.es

#? 4
Nombre:maría
Teléfono:2345
Mail:maria@ucm.es

#? 2
Buscar:juan
Nombre:juan
Teléfono:2345
Mail:juan@ucm.es

#? 1
Nombre:juan
Teléfono:2345
Mail:juan@ucm.es

Nombre:maría
Teléfono:2345
Mail: maria@ucm.es
```

Notas sobre la implementación:

- Usar una función que imprima el menú y ejecute cada acción. La función debe usar las construcciones `select` y `case`. Ejemplo:

```

select opt in "Ficheros" "Fecha" "Salir"
do
    case $opt in
        "Ficheros")
            ls -l
            ;;
        "Fecha")
            date
            ;;
        "Salir")
            echo "Exiting..."
            break
            ;;
        *)
            echo "Error en opción"
            ;;
    esac
done

```

- Las acciones de la agenda se pueden implementar con los comandos de los anteriores ejercicios.
- Para leer la entrada usar el comando `read` de bash. Por ejemplo, `read NOMBRE` lee de la entrada estándar y guarda la cadena leída en la variable `NOMBRE`.

API del Sistema Operativo

Compilación y depuración de programas.

Ejercicio 1. Considerar el siguiente programa:

```
#include <stdio.h>

char * init_string() {
    char s[] = "Esto es una cadena";
    return s;
}

int main() {
    char * msg = init_string();
    printf("Cadena: %s\n", msg);
    return 0;
}
```

- Compilar el programa con las opciones `-g` (símbolos de depuración) y `-o` (especificar salida). Ejemplo: `gcc -g -o error error.c`. Observar los posibles avisos por parte del compilador (*warnings*). Ejecuta el programa y comprueba su funcionamiento.
- Depura el programa, con el comando `gdb ./error` (observa cómo cambia al *prompt* interactivo del depurador):
 - Consulta el código del programa, **list**
 - Añade un punto de parada en la primera línea del programa, **break 10**
 - Ejecuta el programa, **run**
 - Consulta el valor de la variable `msg`, **print msg**
 - Entra en la función (**step**) y avanza hasta la sentencia `return s` (**next**). Consulta el valor de `s` antes de retornar, **print s**.
 - Continúa trazando el programa hasta la línea `printf("...")`. Consulta el valor de `msg` en ese momento.
- Identifica el problema y arregla la función `init_string` para que el código muestre el resultado esperado.

Gestión de errores

Ejercicio 2. Considera el siguiente programa:

```
#include <fcntl.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int fd = open(argv[1], O_RDONLY);
    /* Gestionar el error de open */
    return 0;
}
```

Añadir el código necesario para gestionar correctamente los errores generados por `open(2)` con

`perror(3)`. Consultar en el manual como `open(2)` señala el error y los posibles errores. Comprobar que se identifica correctamente el error en los siguientes casos:

- No se especifica argumento: `./mi_open`
- Se usa una ruta de un fichero que no existe: `./mi_open /no/existe`
- Se usa la ruta de un fichero sin permiso de acceso: `./mi_open /etc/shadow`

Ejercicio 3. Imprimir el código numérico de error generado por la llamada del código anterior (`errno` definido en `errno.h`) y el mensaje asociado obtenido con `strerror(3)`.

Librerías y Llamadas al Sistema

Ejercicio 1. La herramienta `strace(1)` muestra qué llamadas al sistema realiza un programa, y es la forma más directa de “ver” el cambio de modo usuario a modo kernel que ocurre en cada una. Ejecutar `strace ./mi_open /etc/passwd` (usando el programa del ejercicio de gestión de errores) y localizar en la salida la línea correspondiente a la llamada `open(2)`. Repetir con una ruta que no exista y comparar qué código de error aparece en la traza de `strace` frente al que imprime el propio programa con `perror`.

Ejercicio 2. Consultar la página de manual del comando `id(1)` y comprobar su funcionamiento.

Ejercicio 2b. Escribir un programa (`mi_id`) que reciba un único argumento que será el ID de un usuario del sistema. Usando la llamada a librería `getpwnid(3)` imprimir el nombre de usuario, grupo principal y directorio `home`:

```
$ ./mi_id 0
Nombre de usuario: root
ID de usuario,grupo: 0,0
Directorio home: /root
$ ./mi_id 89
Usuario no existe
```

Ejercicio 3. Modificar el programa anterior para que si no se usa ningún argumento se obtenga el ID del proceso con `getuid(2)` y muestre la información asociada a ese usuario.

```
$ ./mi_id
Nombre de usuario: user0S
ID de usuario,grupo: 1000,1000
Directorio home: /home/user0S
```

Ejercicio 4. Consultar la página de manual de `uname(1)` y obtener información del sistema.

Ejercicio 5. Escribir un programa (`mi_uname`) que muestre usando la llamada al sistema `uname(2)` la información del sistema.

```
$ ./mi_uname
Nombre SO: Linux
Version SO: 6.12.44_1 - #1 SMP PREEMPT_DYNAMIC
Nombre Host: pc-user0S
Arquitectura: x86_64
```